

Lecture 3

ML Parameters Settings and Feature Engineering

Course overview

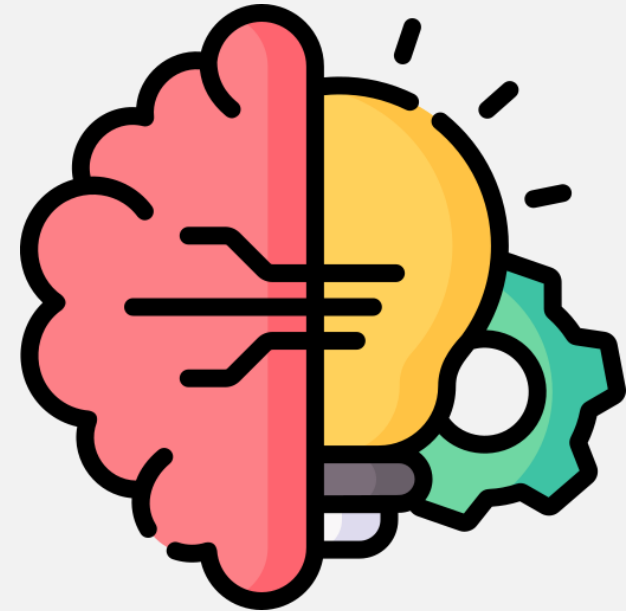
- **Learning Objectives**

- Introduce and familiarise you with the hyperparameters: capacity and Optimization
- Understand Exploratory Data Analysis EDA.
- Build understanding about practical usage of Kernel—based model SVM model.



Table's Contents

- Loss Functions - continue
- Hyperparameters: capacity and Optimization
- Exploratory Data Analysis EDA
- Feature Analysis
- Kernel—based model SVM



Note: majority of the slides have been considered from Prof. Naeem Ullah Khan et al. lectures except few slides.

Recap - Loss Functions

Why Squared Error?

Problem with MSE

MSE measures **numeric distance**, but in classification, we care about **probability confidence**, not distance!

What We Really Want

A loss function that **heavily penalizes confident wrong predictions**

Example: Classifying an image as a dog with 98% confidence when it's actually a cat should incur massive penalty!

Solution: The Logarithm!

The logarithm function has exactly the property we need:

it grows exponentially as confidence in the **wrong** class increases (**This will continue on Day3**)

Recap - Loss Functions

Why Squared Error?

Problem with MSE

MSE measures **numeric distance**, but in classification, we care about **probability confidence**, not distance!

What We Really Want

A loss function that **heavily penalizes confident wrong predictions**

Example: Classifying an image as a dog with 98% confidence when it's actually a cat should incur massive penalty!

Solution: The Logarithm!

The logarithm function has exactly the property we need:
it grows exponentially as confidence in the **wrong** class increases

The Power of the Logarithm

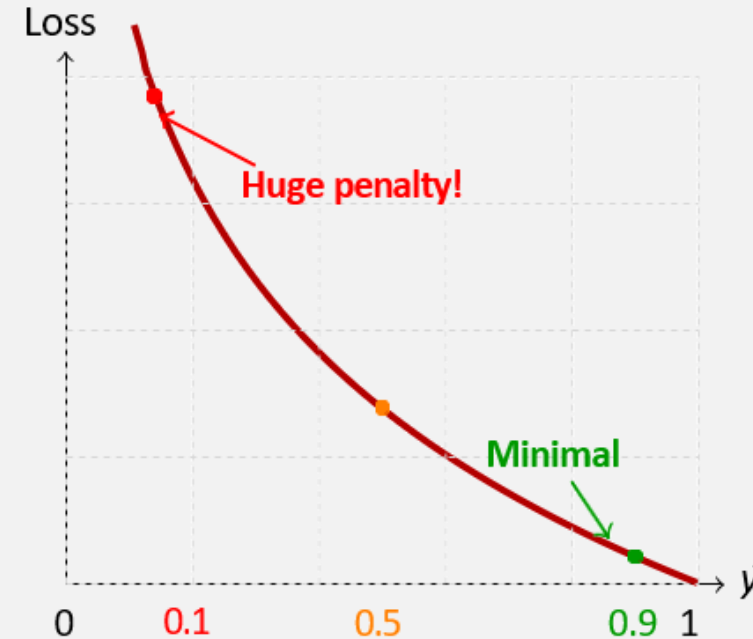
Intuition (True Label $y = 1$):

Model outputs probability $\hat{y}$ for class 1

Log Penalty = $-\log(\hat{y})$ behaves as:

- ▶ $\hat{y} = 0.99 \rightarrow \text{Loss} \approx 0.01$ ✓ **Great!**
- ▶ $\hat{y} = 0.50 \rightarrow \text{Loss} \approx 0.69$ **Uncertain**
- ▶ $\hat{y} = 0.01 \rightarrow \text{Loss} \approx 4.6$ ✗ **Disaster!**

Loss vs. Predicted Probability
(when true label = 1)



Curve explodes as $\hat{y} \rightarrow 0$
Forces model to be cautious!

Binary Cross-Entropy Loss (Log Loss)

Loss Function: Binary Cross-Entropy (Log Loss)

$$J(\mathbf{w}) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

where: $y_i \in \{0, 1\}$ = true label, $\hat{y}_i \in (0, 1)$ = predicted probability, $\mathbf{w}$ = model weights.

- ▶ When $y_i = 1$: Loss = $-\log(\hat{y}_i)$ (penalizes low probability for class 1)
- ▶ When $y_i = 0$: Loss = $-\log(1 - \hat{y}_i)$ (penalizes high probability for class 1)
- ▶ Both terms encourage the model to predict probabilities close to the true labels.

But What If We Have More Than Two Classes?

Binary Classification:

- ▶ $y \in \{0, 1\}$
- ▶ Two classes only
- ▶ Example: Spam / Not Spam

Multiclass Classification:

- ▶ $y \in \{0, 1, 2, \dots, K - 1\}$
- ▶ K classes ($K > 2$)
- ▶ Example: Dog / Cat / Bird

Example: Image Classification

Input: Image $\rightarrow$ Output: $\hat{y} = [0.1, 0.7, 0.2]$ for 3 classes

Class probabilities: Dog (10%), Cat (70%), Bird (20%)

Model now outputs a **probability distribution** over all K classes instead of a single probability

Cross-Entropy (Categorical Cross-Entropy)

Loss Function: Cross-Entropy

Generalization of binary cross-entropy to K classes

$$J(\mathbf{w}) = -\frac{1}{n} \underbrace{\sum_{i=1}^n}_{\text{samples}} \underbrace{\sum_{k=1}^K}_{\text{classes}} y_{i,k} \log(\hat{y}_{i,k})$$

where $y_{i,k} \in \{0, 1\}$ is 1 if sample i belongs to class k , 0 otherwise

How It Works:

- ▶ Model outputs probabilities for all K classes: $\hat{\mathbf{y}}_i = [\hat{y}_{i,1}, \hat{y}_{i,2}, \dots, \hat{y}_{i,K}]$
- ▶ True label is one-hot encoded: $\mathbf{y}_i = [0, 1, 0, \dots, 0]$ if class 2
- ▶ Loss penalizes the probability assigned to the **true class**

Cross-Entropy (Categorical Cross-Entropy)

Loss Function: Cross-Entropy

Generalization of binary cross-entropy to K classes

$$J(\mathbf{w}) = -\frac{1}{n} \underbrace{\sum_{i=1}^n}_{\text{samples}} \underbrace{\sum_{k=1}^K}_{\text{classes}} y_{i,k} \log(\hat{y}_{i,k})$$

where $y_{i,k} \in \{0, 1\}$ is 1 if sample i belongs to class k , 0 otherwise

Example

True class: Cat (class 2), Prediction: [0.1, 0.7, 0.2]

Loss = $-\log(0.7) \approx 0.36$ (only the true class matters!)

By now, we covered the Loss Functions
from Three Pillars of learning

Hyperparameters: Capacity & Optimization

Hyperparameters are settings chosen **before** training.

- **Structural:** Control model complexity/capacity (e.g., Tree Depth, Layers).
- **Optimization:** Control the training process (e.g., Learning Rate).

A. Structural Params (Controlling Overfitting)

Rigid (Risk: Underfitting)

- ▶ Poly Degree = 1 (Line)
- ▶ Tree Depth = 1 (Stump)
- ▶ NN Layers = 1 (Shallow)

Flexible (Risk: Overfitting)

- ▶ Poly Degree = 20 (Wiggly)
- ▶ Tree Depth = 100 (Deep)
- ▶ NN Layers = 20 (Deep)

Hyperparameters: Capacity & Optimization

Hyperparameters are settings chosen **before** training.

- **Structural:** Control model complexity/capacity (e.g., Tree Depth, Layers).
- **Optimization:** Control the training process (e.g., Learning Rate).

B. Optimization Params (The "Learning" Process)

Learning Rate (LR): The "Step Size" of the update.

Too Low
→ Slow/Stuck

Just Right
→ Converges

Too High
→ Unstable/Diverges

Hyperparameters: Capacity & Optimization

Hyperparameters are settings chosen **before** training.

- **Structural:** Control model complexity/capacity (e.g., Tree Depth, Layers).
- **Optimization:** Control the training process (e.g., Learning Rate).

B. Optimization Params (The "Learning" Process)

Learning Rate (LR): The "Step Size" of the update.

Too Low
→ Slow/Stuck

Just Right
→ Converges

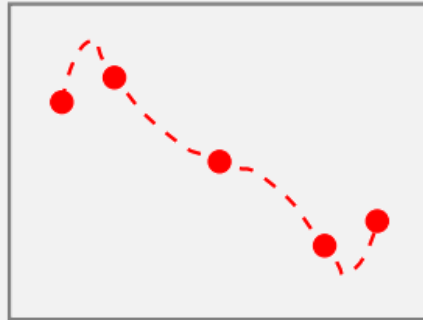
Too High
→ Unstable/Diverges

Increase Data Size

The Logic

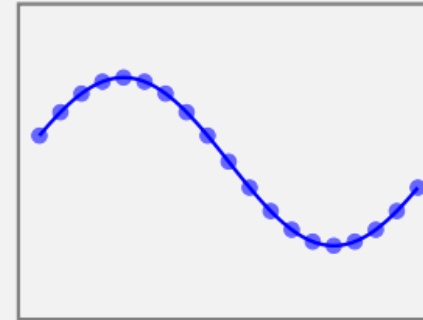
Overfitting happens when the model memorizes "noise" in the gaps between data points. **Solution:** Fill the gaps with more data

1. Small Data (Sparse)



Easy to memorize path

2. Large Data (Dense)



Forced to see the curve

With more data, the "wiggly" memorization path becomes impossible. The model **must** learn the true underlying curve (Signal).

Reduce Features

The Curse of Dimensionality

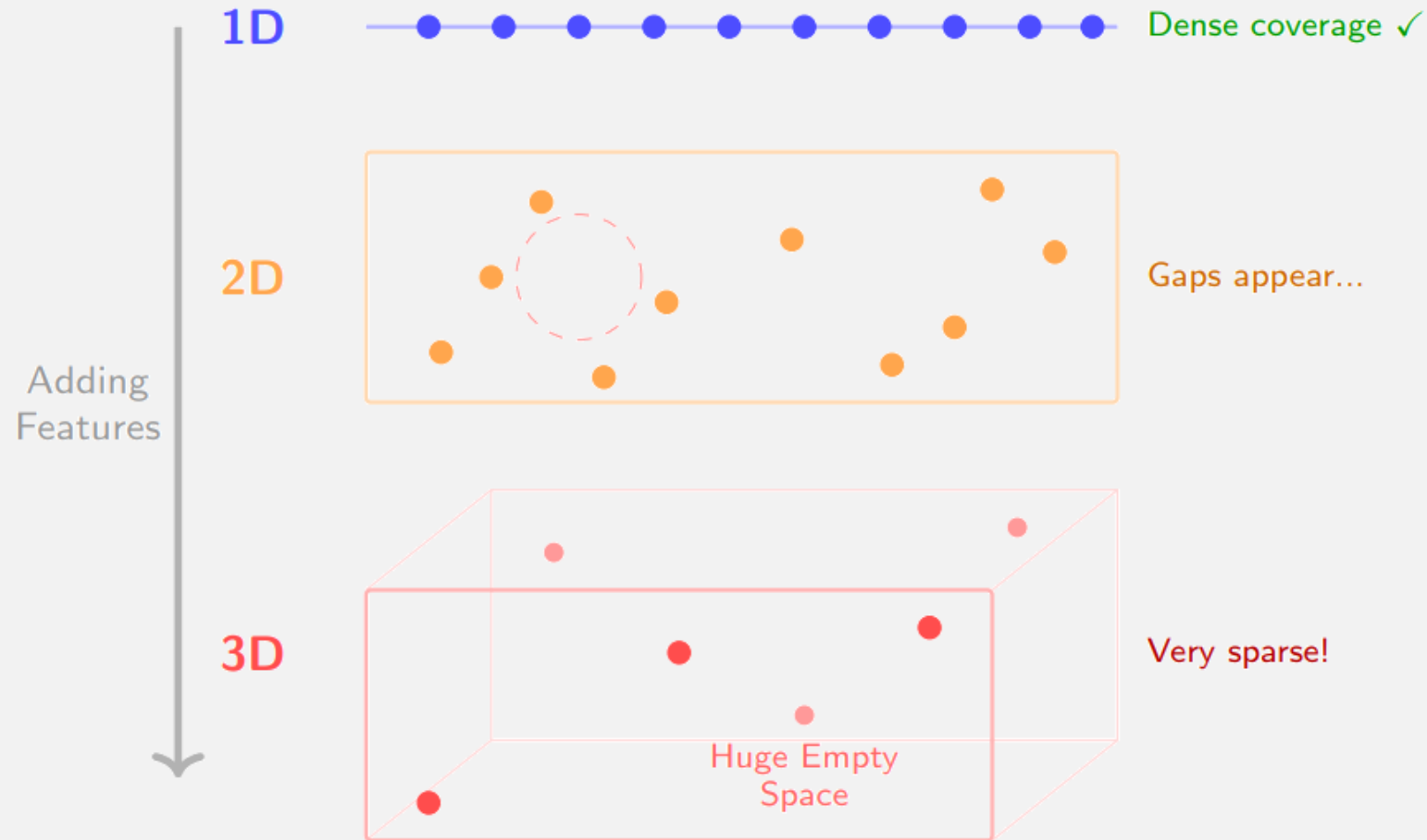
As you add more features (dimensions), the space grows exponentially.

Your data becomes sparse, creating empty regions where the model

hallucinates patterns.

The curse of Dimensionality

Visualizing the problem: Same 10 samples, increasing dimensions.



The curse of Dimensionality

What Happens in High Dimensions?

- **1D:** 10 points densely cover the line.
- **2D:** Same 10 points spread thin → gaps appear.
- **3D:** Mostly empty space → the model starts to find fake patterns in the noise just to bridge the gaps.

The curse of Dimensionality

What Happens in High Dimensions?

- **1D**: 10 points densely cover the line.
- **2D**: Same 10 points spread thin → gaps appear.
- **3D**: Mostly empty space → the model starts to find fake patterns in the noise just to bridge the gaps.

The Fix: Feature Selection

- ▶ Remove irrelevant features (noise) and keep only meaningful signals.
- ▶ This should push the data back down to lower dimensions where density is high.

Summary: Diagnosing and Fixing

Problem	Diagnosis (Loss)	Mitigation (Fix)
Underfitting	Train High, Val High	↑ Increase Complexity ↑ Add Features
Overfitting	Train Low, Val High	↓ Reduce Complexity ↓ Remove Noise/Features ↑ More Data → Regularization (Next Days)
Good Fit	Train Low, Val Low	✓ All good!

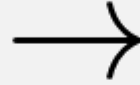
From Theory to Practice

We know that **Good Data** cures Overfitting. But real-world data is rarely "Good." It is messy, broken, and chaotic

Raw Data (Reality)

```
["NaN", "N/A", -500,  
 "Cat"]
```

Models crash on this.



Clean Data

```
[0.5, 0.2, 0.0, 1.0]
```

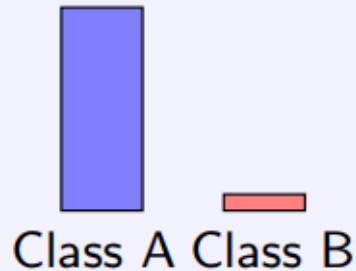
Models learn from this.

Step1: Exploratory Data Analysis EDA

Goal

Before training, become a Detective. If you feed garbage in, you get garbage out.

1. Check Target Distribution



- ▶ Is it Imbalanced?
- ▶ If yes, use Stratified Split & F1-Score.

2. Sanity Checks (Red Flags)

- ▶ **Impossible Values:**
- ▶ Age: -5 → **Error?**
- ▶ Age: 200 → **Outlier?**
- ▶ **Leakage:**
- ▶ Does column X perfectly predict y? (e.g., "TransactionID" contains the label).

Step2: Cleaning and Scaling

A. Missing Values (NaN) Models cannot do math on "Empty".

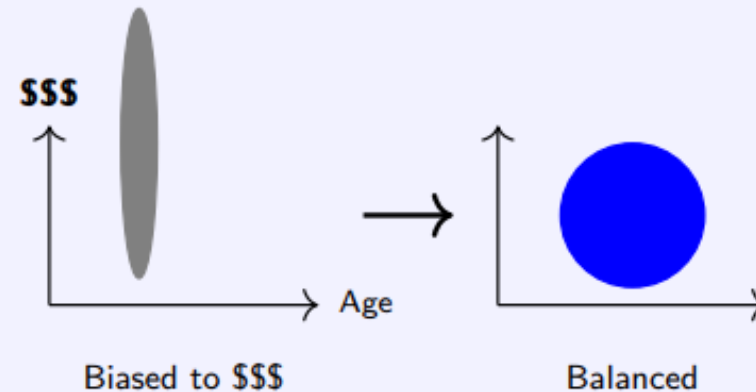
Strategies

1. **Drop:** Delete the row (only if you have lots of data).
2. **Impute:** Fill with Mean/Median.

$[10, ?, 30] \rightarrow [10, 20, 30]$

B. Feature Scaling Income (100k) vs. Age (50).

Why Scale?



Models prioritize larger numbers. Scale to $[0, 1]$ (Min-Max Scaler) or $\mu = 0, \sigma = 1$ (Standard Scaler).

Step3: Encoding and Feature Engineering

C. Encoding (Text → Numbers)

Models speak Math, not English.

1. Label Encoding (Ordinal)

- ▶ ["Low", "High"] → [0, 1]
- ▶ Use if order matters.

2. One-Hot Encoding (Nominal)

- ▶ ["Cat", "Dog"] →
- ▶ Cat: [1, 0], Dog: [0, 1]
- ▶ Use if no order exists.

D. Feature Engineering Creating new signals from existing data.

Example: Date Object

"2023-12-25"

↓ *Extract Info* ↓

- ▶ Is_Weekend = 0
- ▶ Month = 12 (Winter)
- ▶ Days_To_Holiday = 0

This adds value that the raw date string hid.

Step4: Fit the model

We have clean data. We have robust splits. We have metrics.

Now, we need a model.

The Options

- **Regression:** Linear Regression, RF Regressor, ..
- **Classification:** KNN, SVM, SVM

**We have covered LR, KNN.
Lets Proceed with SVM.**

Support Vector Machine

What is SVM?

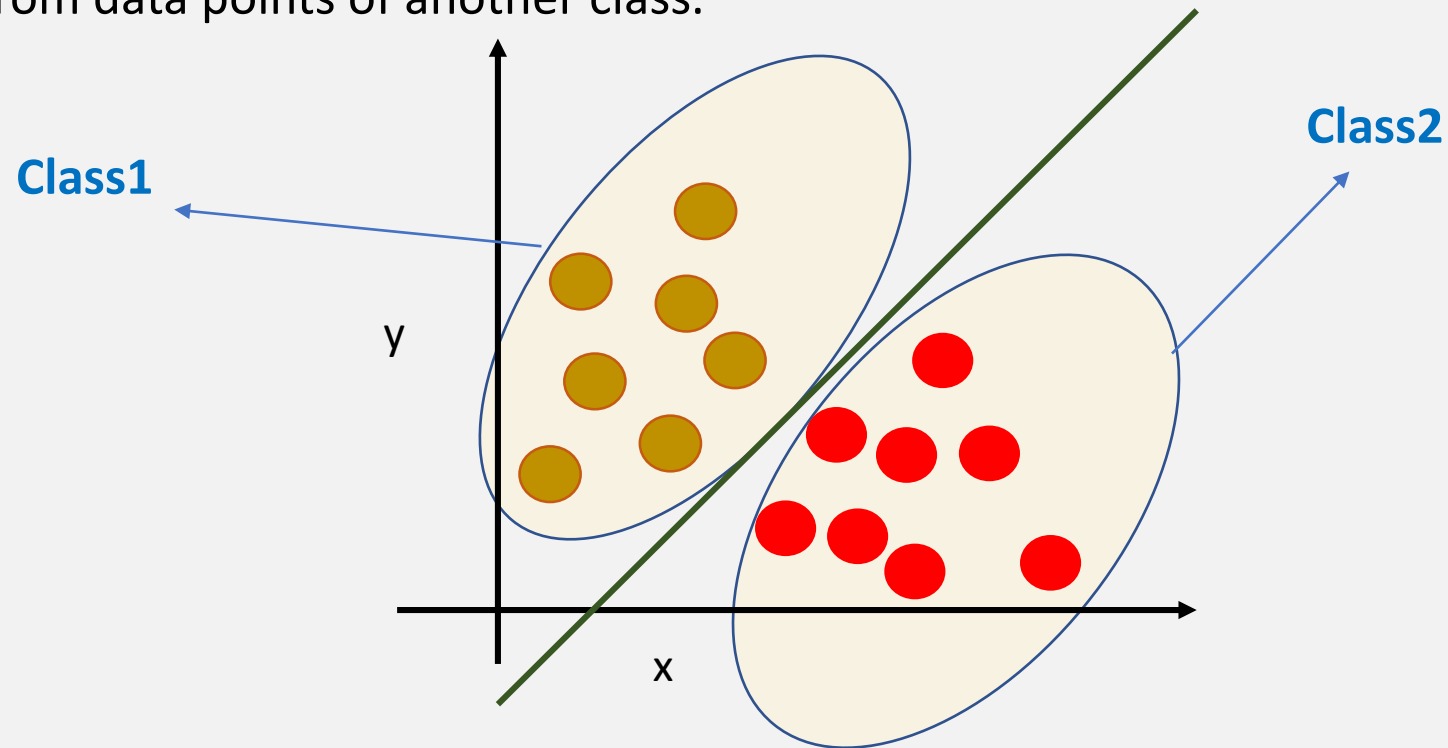
Developed in the 1990s by Vladimir N. Vapnik and colleagues

- A powerful machine learning algorithm widely used for both **linear** and **nonlinear** classification, as well as regression and outlier detection tasks.
- SVMs are highly adaptable, making them suitable for various applications such as
 - Text classification, image classification, spam detection, handwriting identification, gene expression analysis, face detection, and anomaly detection.

Support Vector Machine

What is SVM?

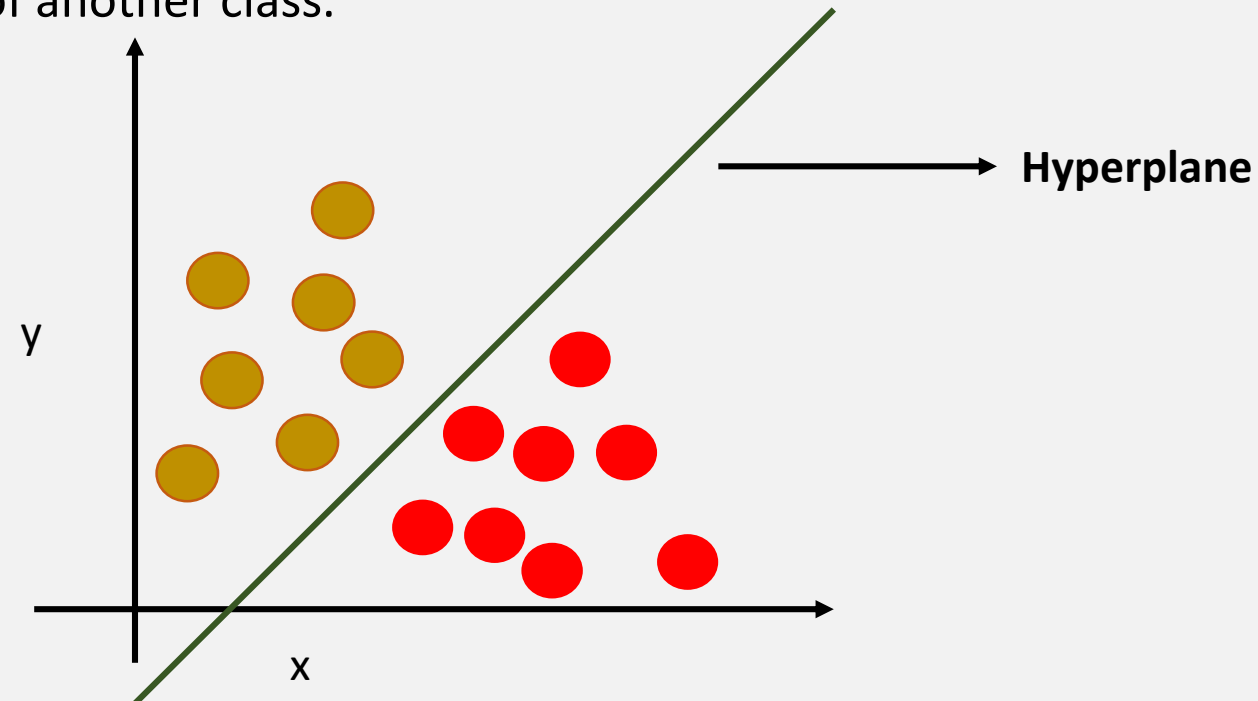
The idea behind SVM is to find a **boundary** (or decision boundary) that best separates data points of one class from data points of another class.



Support Vector Machine

What is SVM?

The idea behind SVM is to find a **boundary** (or decision boundary) that best separates data points of one class from data points of another class.



Support Vector Machine

Scenario: Email Spam Detection

Imagine you building a system to classify (**Spam, not spam**)

Algorithms to classify incoming emails as either **spam** or **not spam**



Not spam



Spam

Data points

Support Vector Machine

Scenario: Email Spam Detection

Goal: Model that can learn to distinguish between these two classes using patterns in past emails.

Features for each email:

X1: Frequency of the word "**free**"

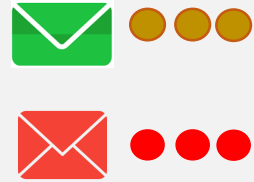
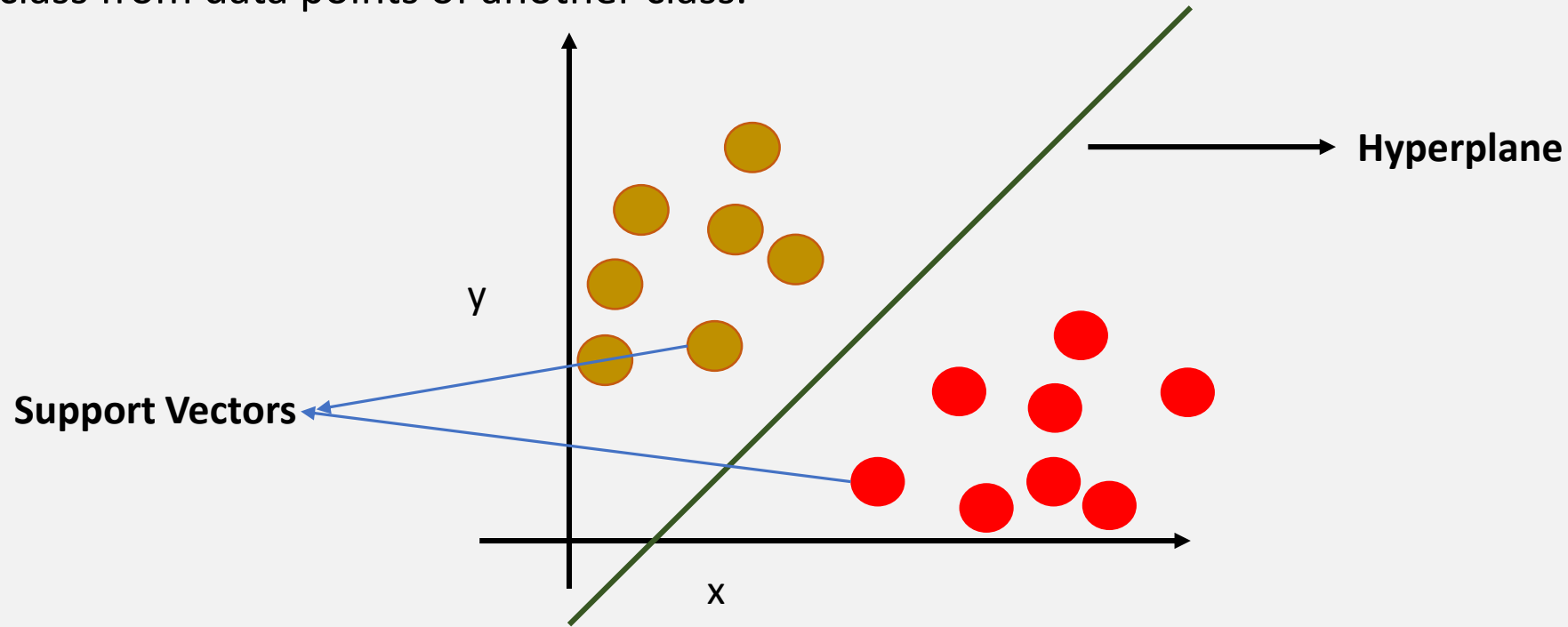
X2: Frequency of the word "**urgent**"

Other potential features: Number of links, sender reputation, presence of certain phrases, etc.

Support Vector Machine

What is SVM?

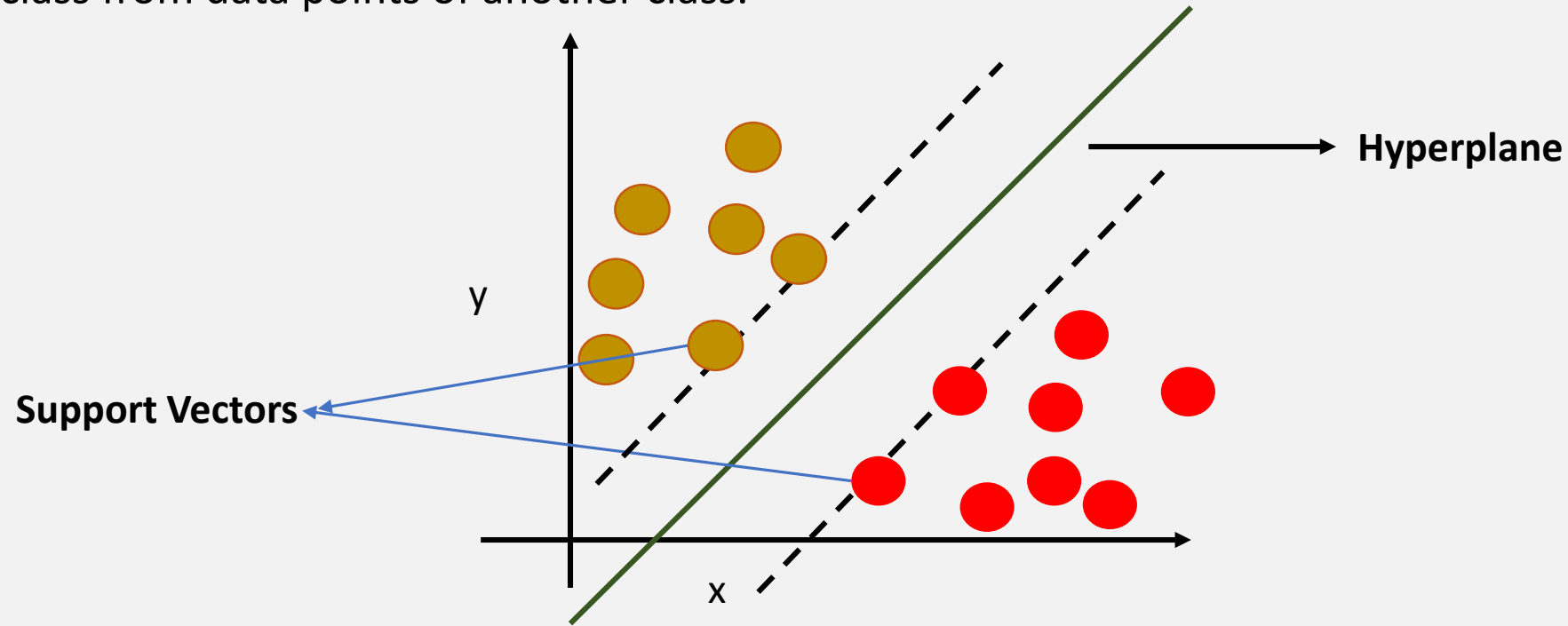
The idea behind SVM is to find a **boundary** (or decision boundary) that best separates data points of one class from data points of another class.



Support Vector Machine

What is SVM?

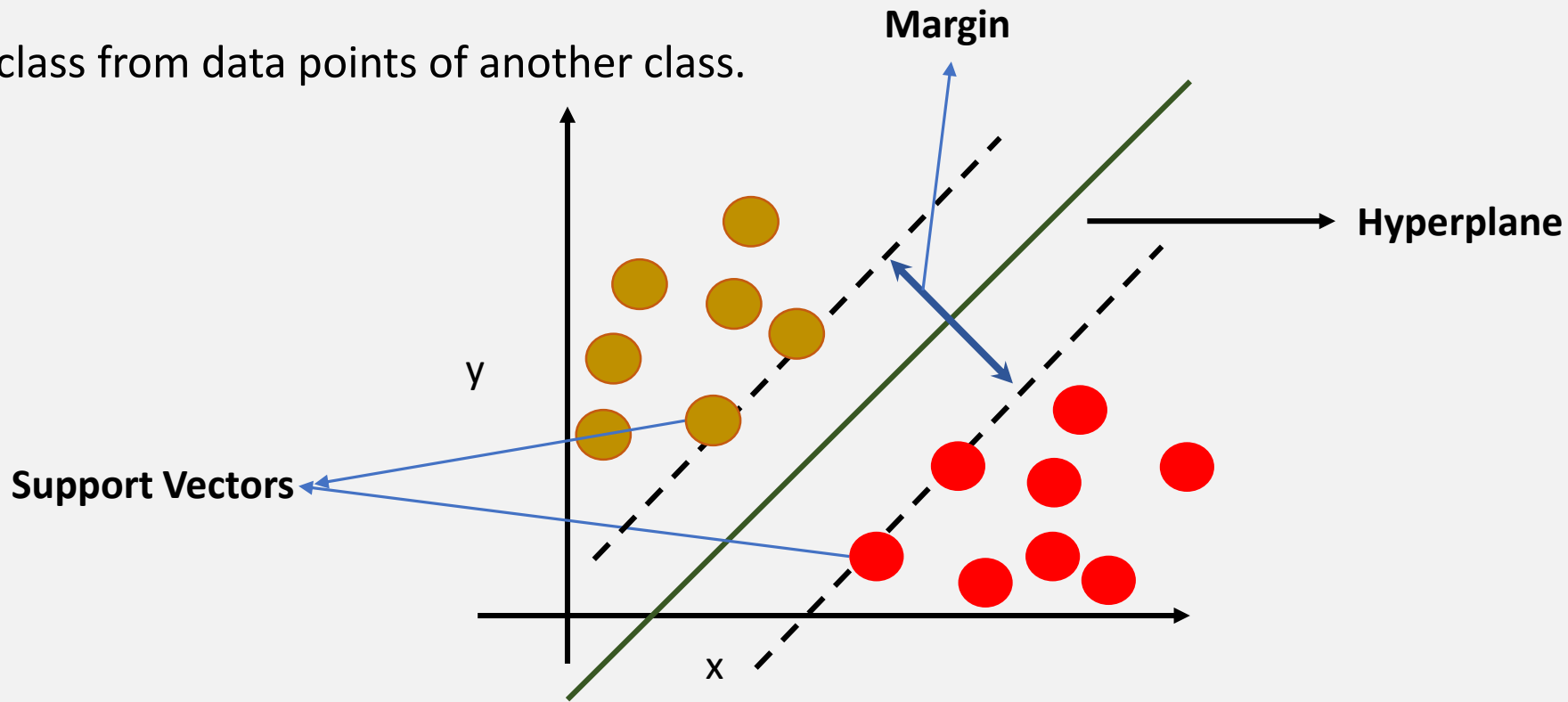
The idea behind SVM is to find a **boundary** (or decision boundary) that best separates data points of one class from data points of another class.



Support Vector Machine

What is SVM?

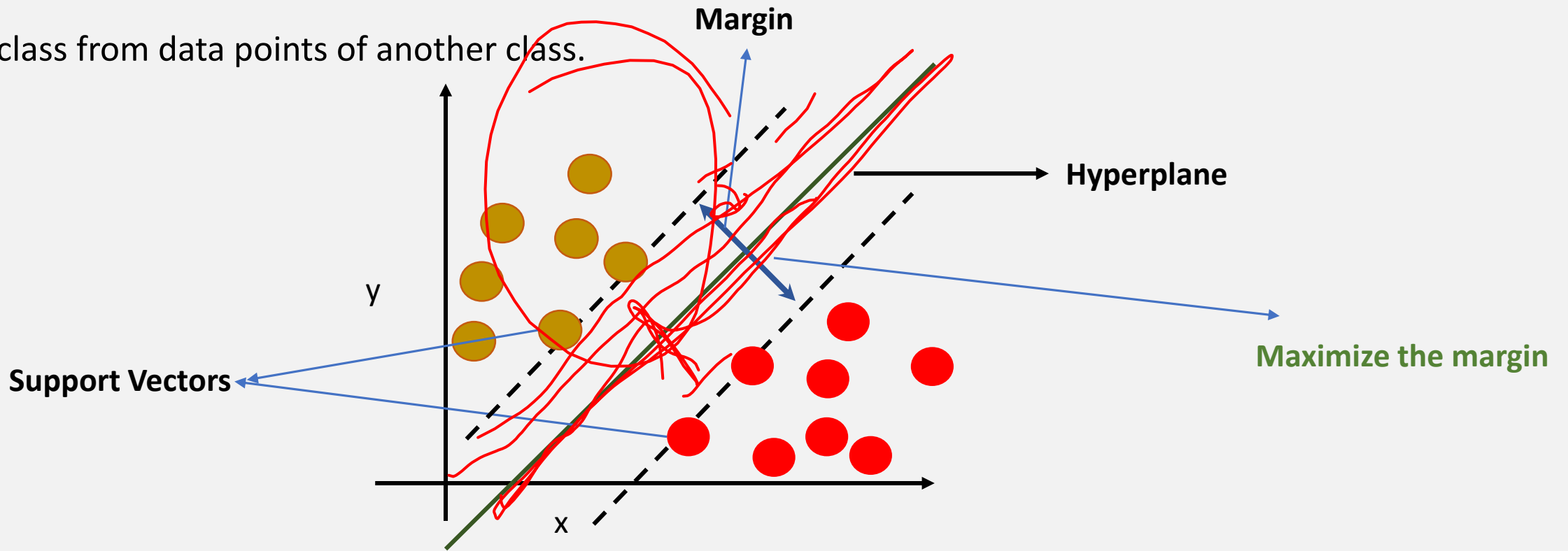
The idea behind SVM is to find a **boundary** (or decision boundary) that best separates data points of one class from data points of another class.



Support Vector Machine

What is SVM?

The idea behind SVM is to find a **boundary** (or decision boundary) that best separates data points of one class from data points of another class.

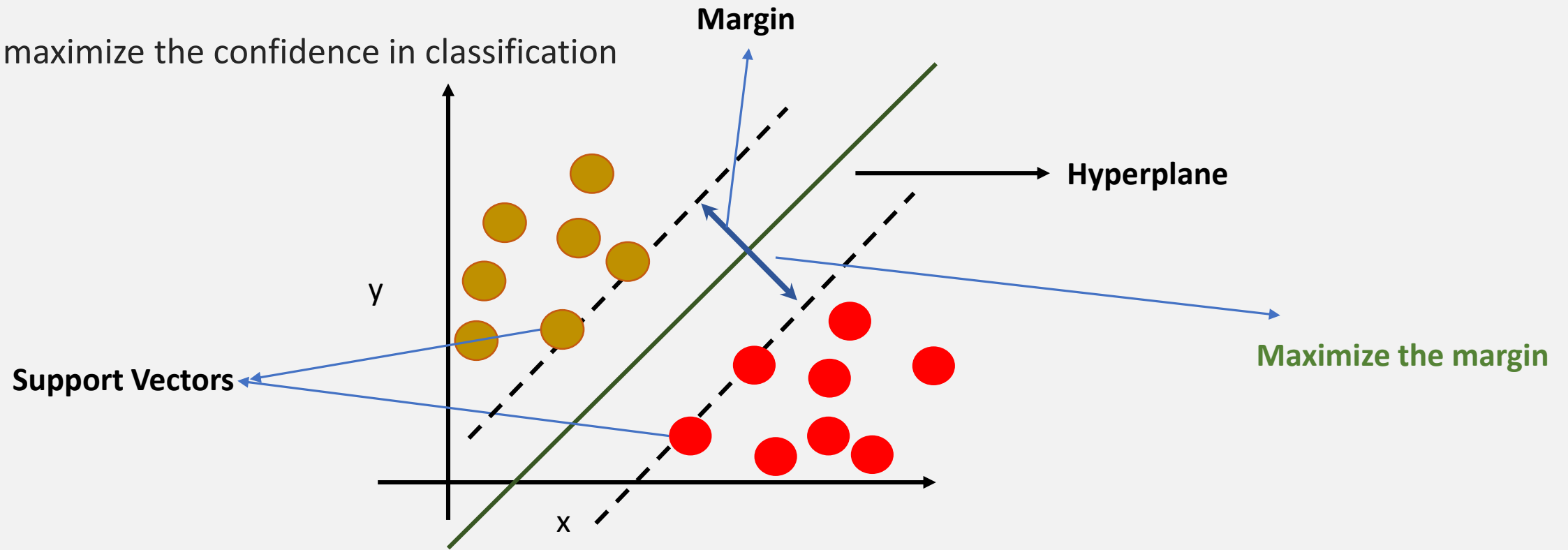


Support Vector Machine

Finding the Optimal Hyperplane

SVM selects the line (hyperplane) that **maximizes the margin** between the two classes (Spam, Not spam).

This maximize the confidence in classification

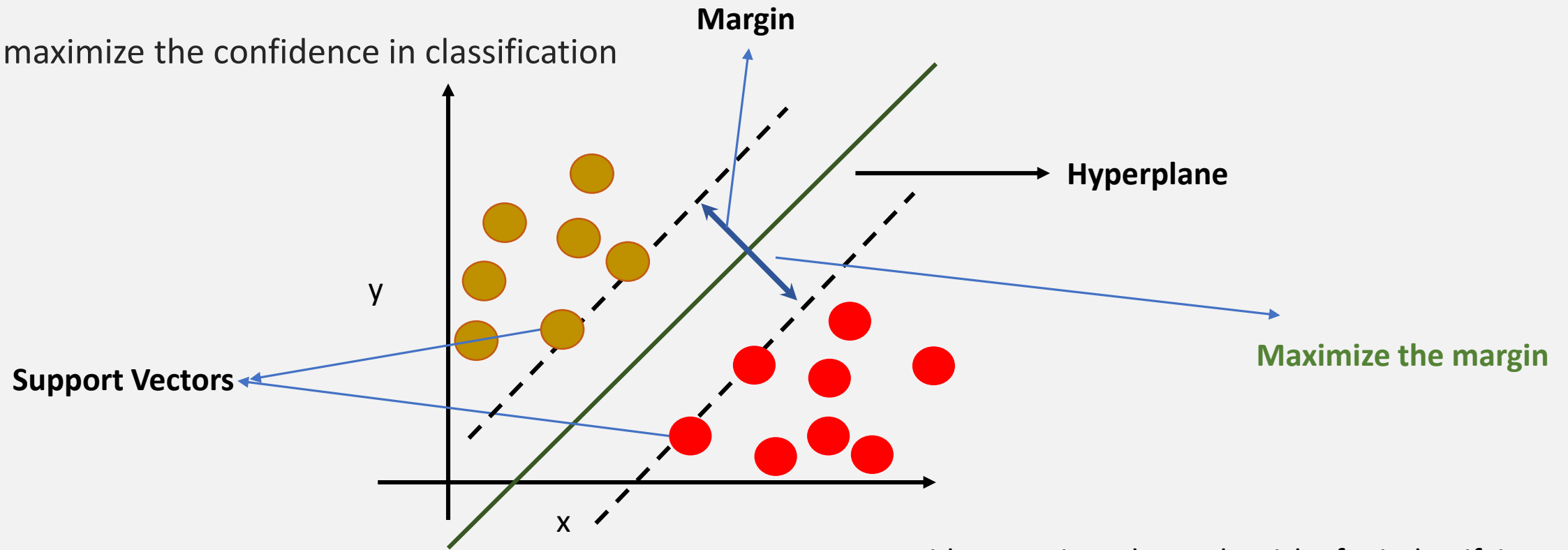


Support Vector Machine

Finding the Optimal Hyperplane

SVM selects the line (hyperplane) that **maximizes the margin** between the two classes (Spam, Not spam).

This maximizes the confidence in classification



Wider margin reduces the risk of misclassifying emails.

Support Vector Machine

Finding the Optimal Hyperplane

Why Maximize the Margin?

- A wider margin reduces the risk of misclassifying emails.
- It helps the model generalize better to new emails.

Equation of Hyperplane

$$w \cdot x + b = 0$$

where $\mathbf{w}$ is the weight vector and b is the bias.

Support Vector Machine

Non-Linearly Separable



Problem with Linear Boundaries:

Facial Expression Recognition: Imagine we're building a model to classify facial expressions into six categories (**Angry**, **Happy**, **Sad**, **Neutral**, **Fear**, **Surprise**).

Each face image can be represented by features like **edge patterns**, **texture**, **brightness**, and **key points on the face** (like the position of eyebrows, mouth, etc.).

Challenge: These six expressions may not be linearly separable due to overlapping or similar features.

For example, "Surprise" and "Fear" might have similar wide-eyed expressions.

Support Vector Machine

Non-Linearly Separable

Solution with Kernels:

SVM can use a *kernel function* to map the data into a higher-dimensional space where it becomes easier to separate.

Radial Basis Function (RBF) Kernel: Useful for non-linear data; helps separate classes by creating a more flexible decision boundary.

One-vs-One Strategy: SVM can classify pairs of expressions at a time. For example, one SVM model might classify "Happy" vs. "Sad," another "Angry" vs. "Neutral," etc., for all combinations.

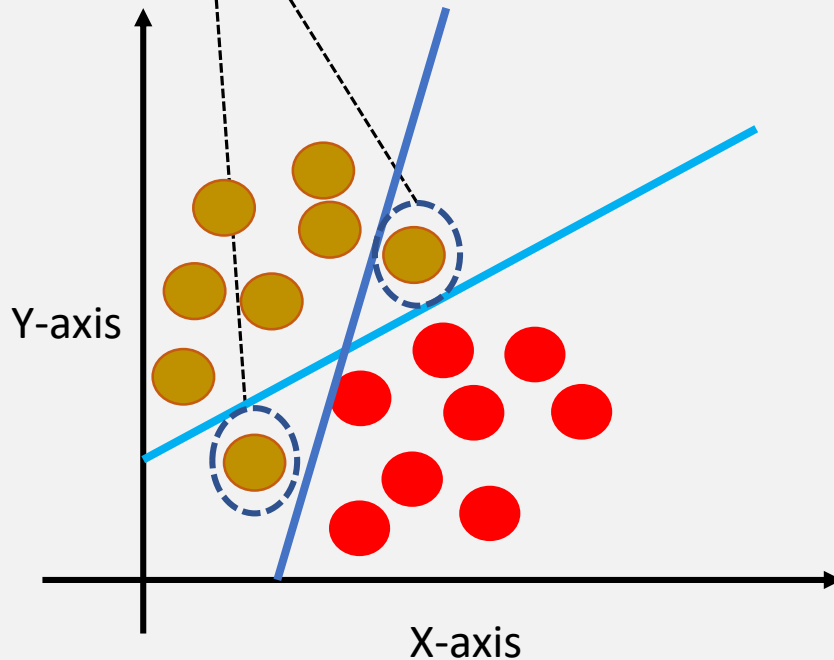
One-vs-All Strategy: Score for each class is computed based on the output of the respective binary classifier that was trained to distinguish that class from all others

Support Vector Machine

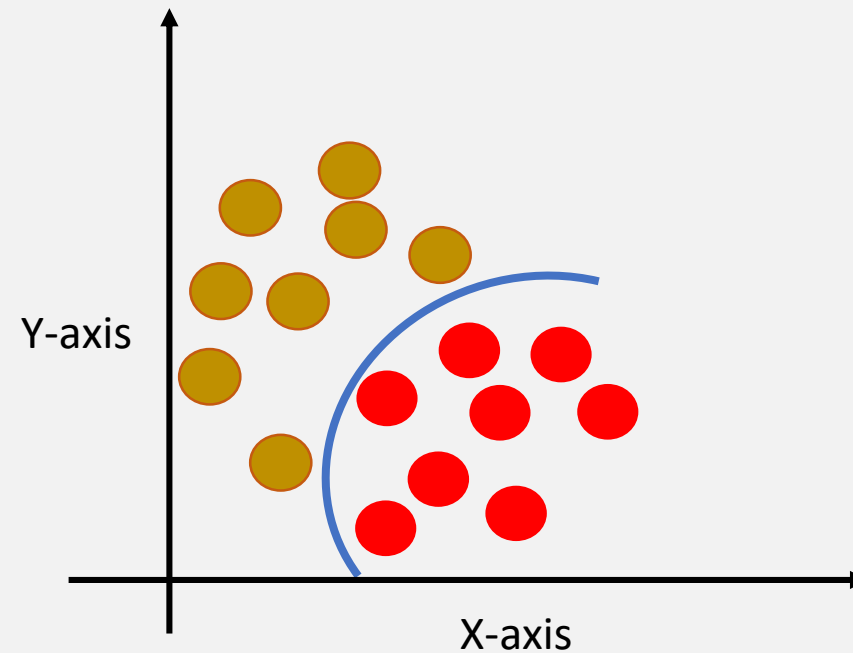
Non-Linearly Separable

Misclassification

RBF maps the features of facial expressions into a higher-dimensional space, making it easier to separate them with curved boundaries.



Using **linear boundary** isn't feasible as the data points overlap



kernel function, like the Radial Basis Function (RBF) kernel.

Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)



SVM Model for Facial Expression Recognition

1. Feature Extraction and Transformation

For new face, key features like **mouth width, eyebrow shapes, eye size, texture patterns**

These features are mapped into higher-dimensional space using RBF kernel so that

model can apply non-linear boundaries

2. Decision Boundaries

SVM uses the transformed data to predict which expression the new face resembles by placing it in one of six classes.

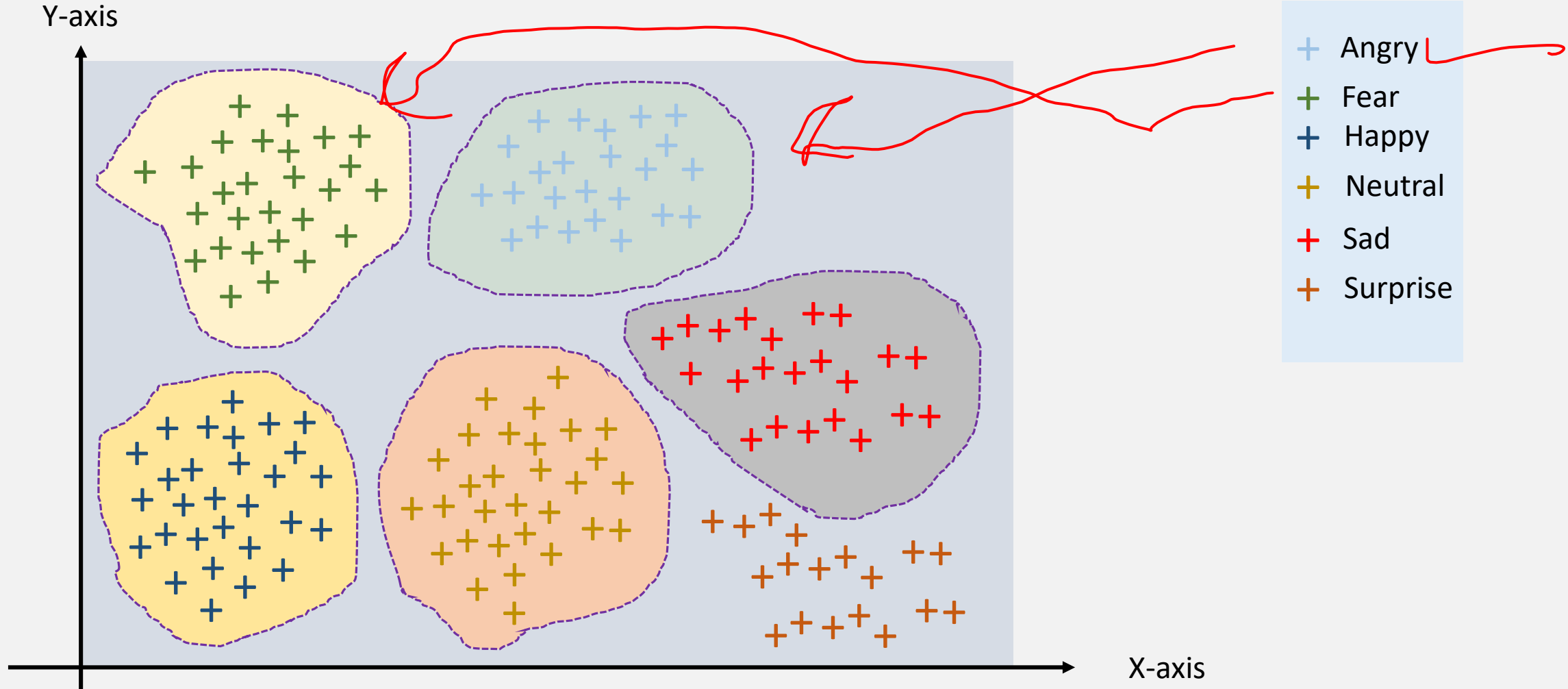
Each SVM model in the **One-vs-One approach** votes for a class, and the majority vote determines the final label.

Score for each class is computed **One-vs-All approach** based on the output of the respective binary classifier that was trained to distinguish that class from all others

Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)

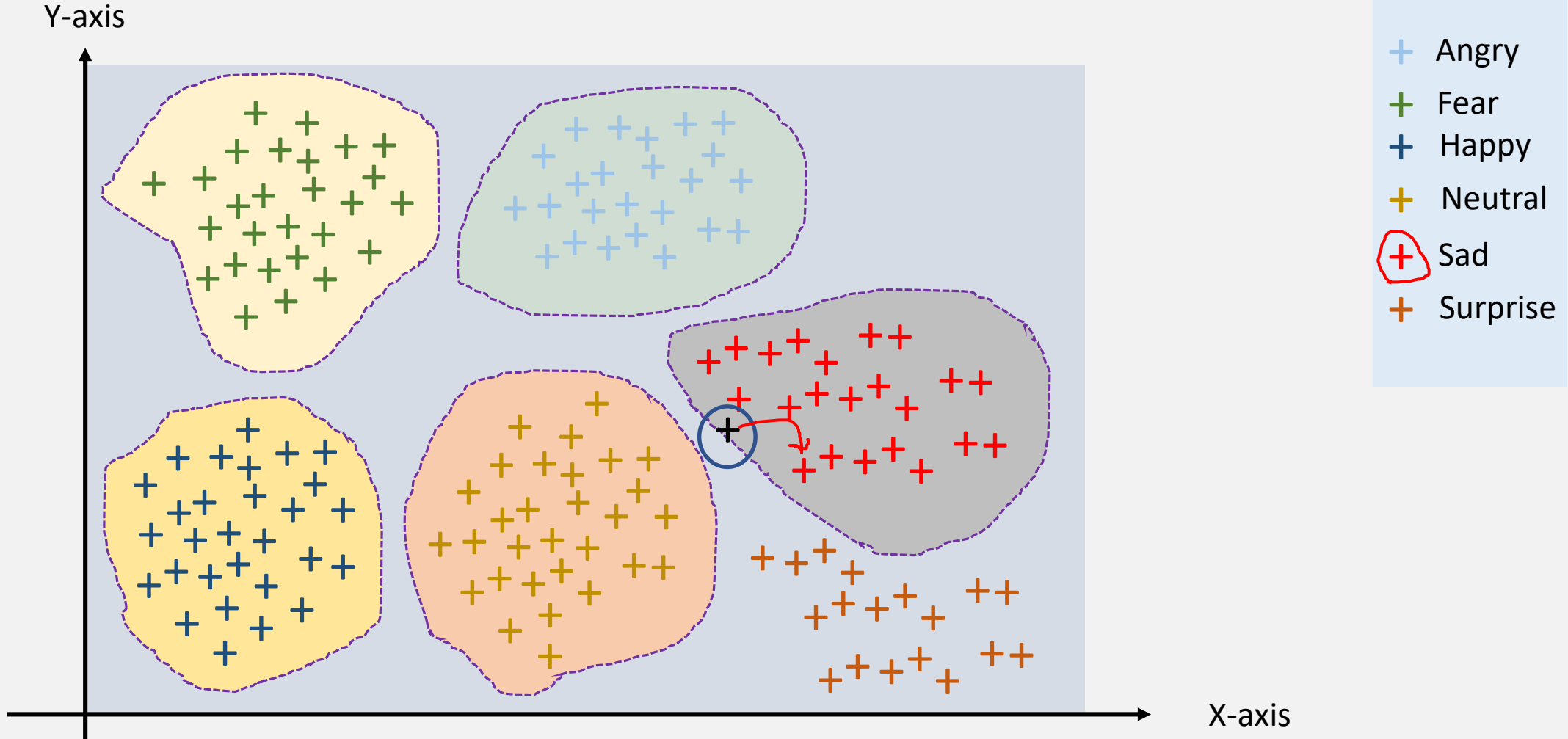
SVM Model for Facial Expression Recognition: One-vs-One approach



Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)

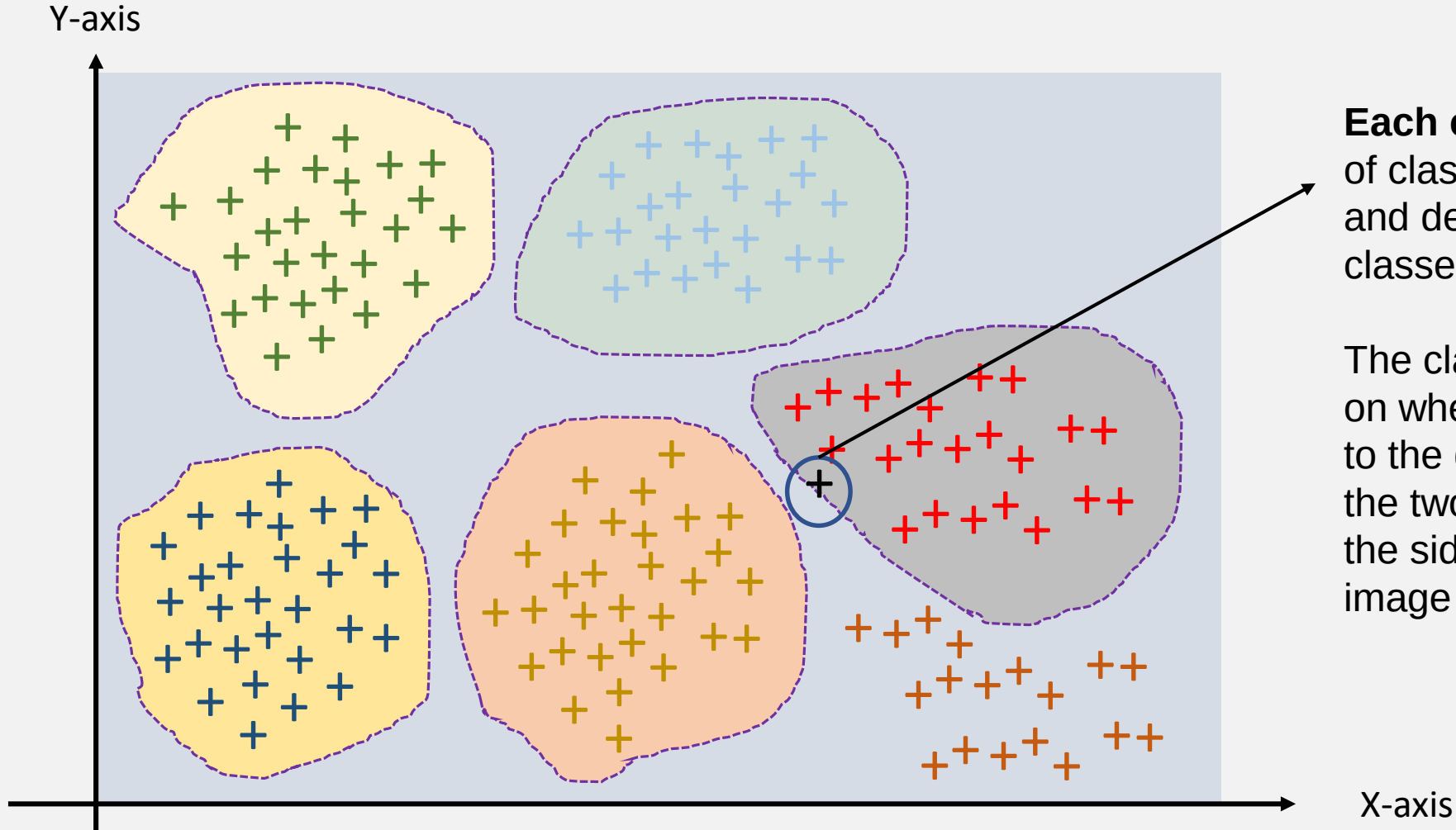
SVM Model for Facial Expression Recognition: One-vs-One approach



Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)

SVM Model for Facial Expression Recognition: One-vs-One approach



Each classifier (trained on a pair of classes) will evaluate the image and decide which of the two classes the image belongs to.

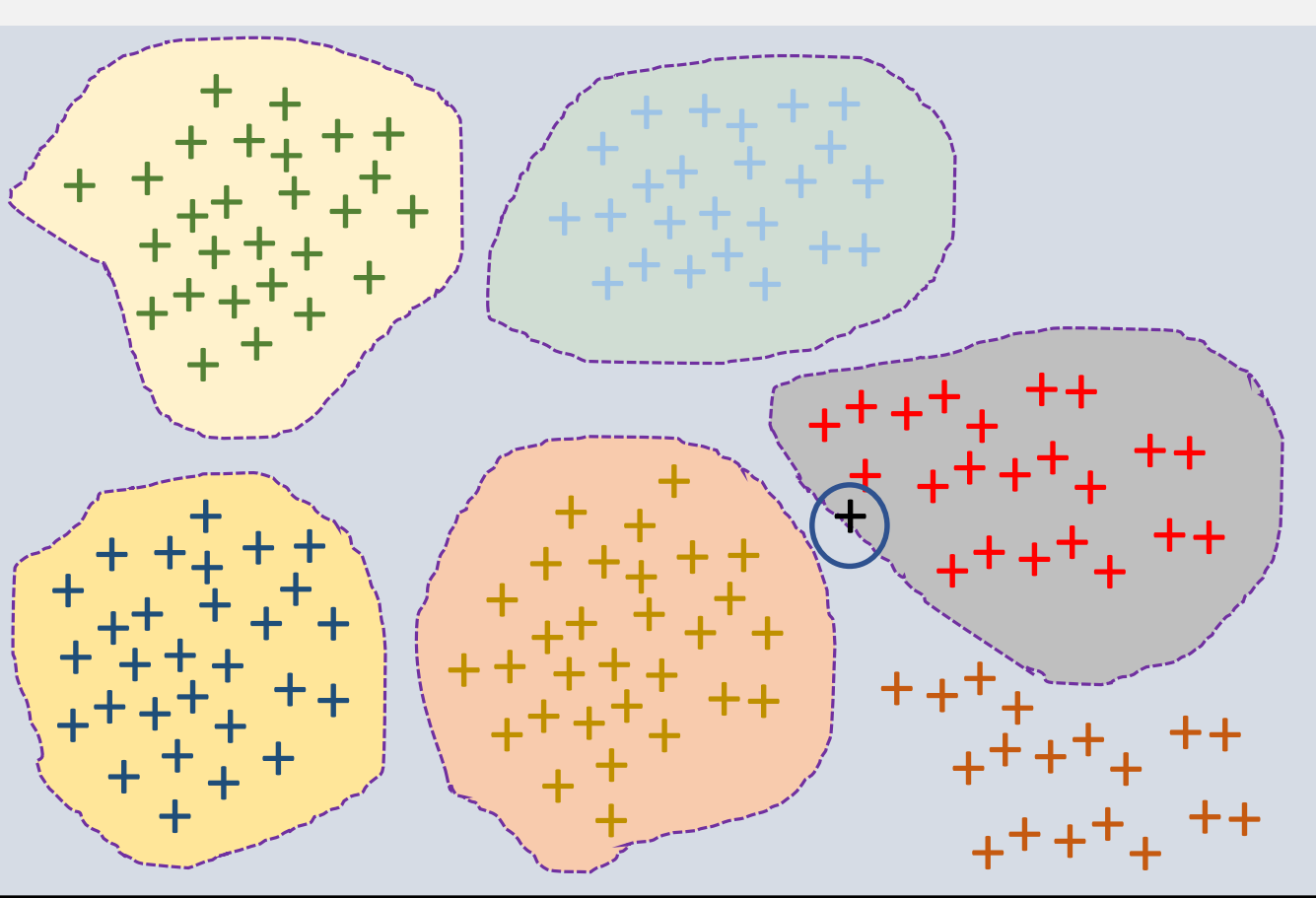
The classifier's decision is based on where the image falls relative to the decision boundary between the two classes. The SVM finds the side of the boundary where the image lies.

Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)

SVM Model for Facial Expression Recognition: One-vs-One approach

Y-axis



Pair (Classifier)	Prediction
Ang Vs Happy	Angry
Ang Vs Sad	Sad
Ang Vs Neutral	Neutral
Ang Vs Fear	Fear
Ang Vs Surprise	Angry
Hap Vs Sad	Sad
Hap Vs Neutral	Neutral
Hap Vs Fear	Happy
Hap Vs Surprise	Surprise
Sad Vs Neutral	Sad
Sad Vs Fear	Sad
Sad Vs Surprise	Sad
Neutral Vs Fear	Fear
Neutral Vs Surprise	Neutral
Fear Vs Surprise	Fear

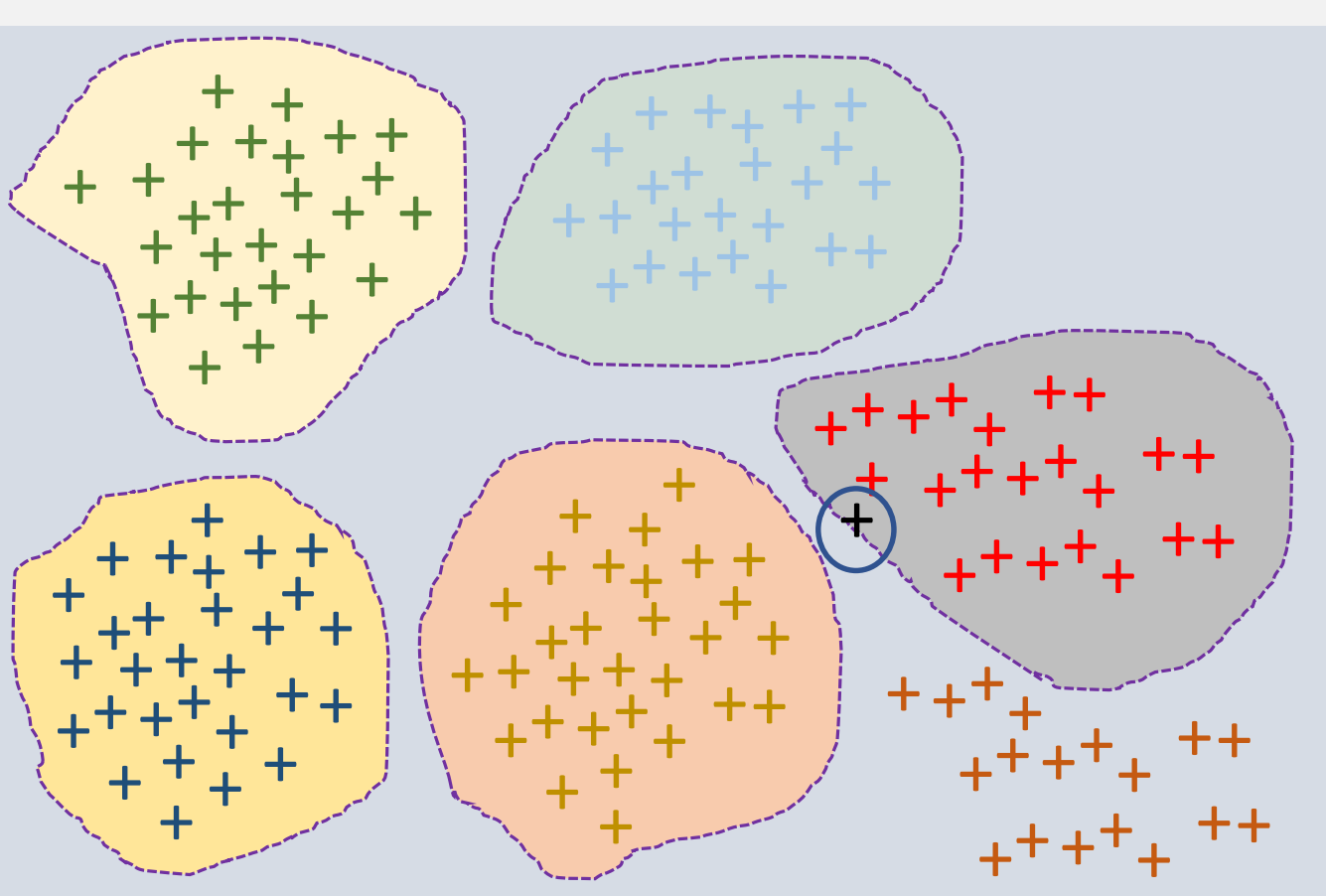
5
Sad

Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)

SVM Model for Facial Expression Recognition: One-vs-One approach

Y-axis



Pair (Classifier)	Prediction
Ang Vs Happy	Angry
Ang Vs Sad	Sad
Ang Vs Neutral	Neutral
Ang Vs Fear	Fear
Ang Vs Surprise	Angry
Hap Vs Sad	Sad
Hap Vs Neutral	Neutral
Hap Vs Fear	Happy
Hap Vs Surprise	Surprise
Sad Vs Neutral	Sad
Sad Vs Fear	Sad
Sad Vs Surprise	Sad
Neutral Vs Fear	Fear
Neutral Vs Surprise	Neutral
Fear Vs Surprise	Fear

3

Fear

3

Neutral

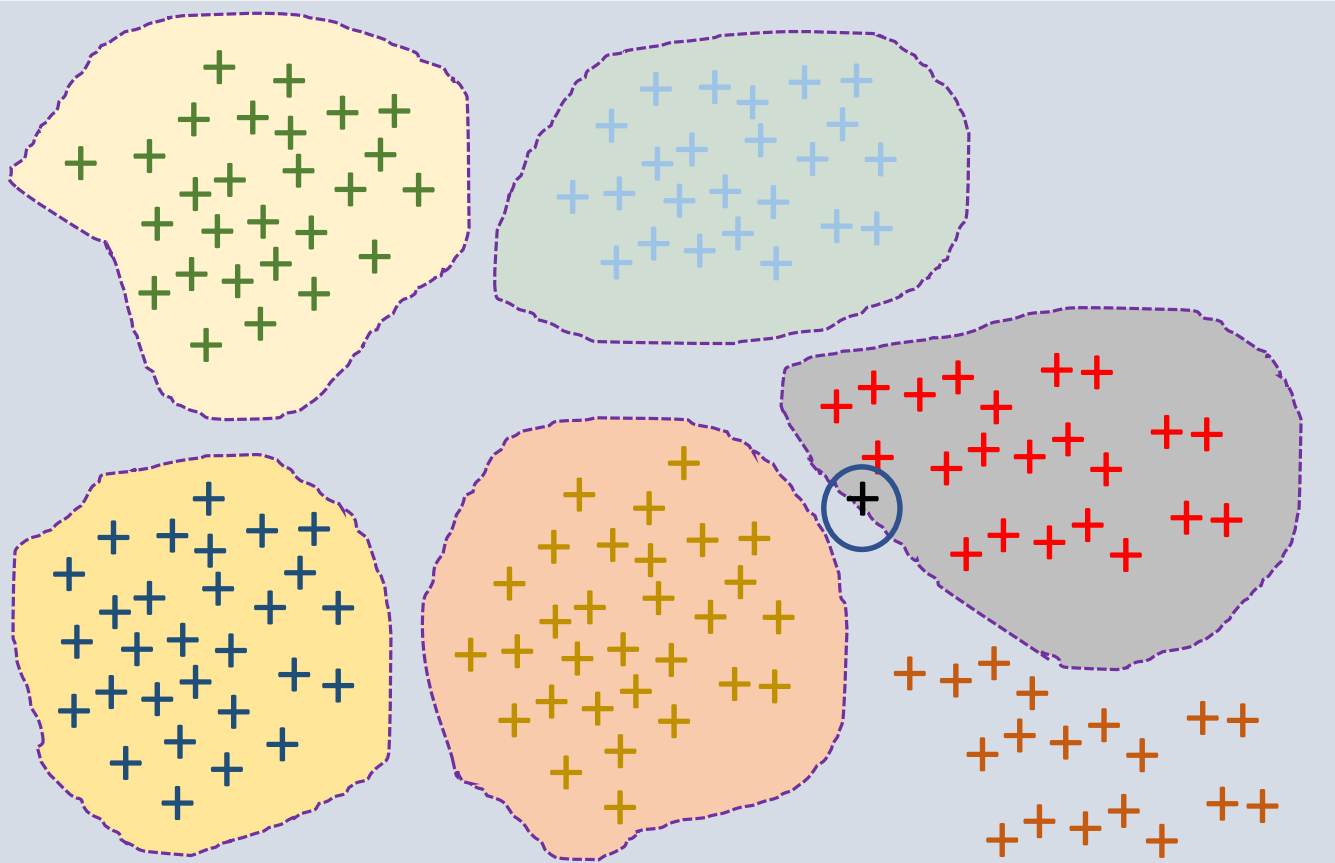
X-axis

Support Vector Machine

(Angry, Fear, Happy, Neutral, Sad, Surprise)

SVM Model for Facial Expression Recognition: One-vs-One approach

Y-axis



Pair (Classifier)	Prediction
Ang Vs Happy	Angry
Ang Vs Sad	Sad
Ang Vs Neutral	Neutral
Ang Vs Fear	Fear
Ang Vs Surprise	Angry
Hap Vs Sad	Sad
Hap Vs Neutral	Neutral
Hap Vs Fear	Happy
Hap Vs Surprise	Surprise
Sad Vs Neutral	Sad
Sad Vs Fear	Sad
Sad Vs Surprise	Sad
Neutral Vs Fear	Fear
Neutral Vs Surprise	Neutral
Fear Vs Surprise	Fear

2
Angry

1
Happy

1
Surprise

X-axis

Support Vector Machine

SVM Model for Facial Expression Recognition: One-vs-Many approach

One classifier per class

We create six separate SVM classifiers.

Each classifier is trained to recognize whether an image **belongs to its specific class or not**

(e.g., Angry vs. not Angry, Happy vs. not Happy).

Training each classifier

For each classifier, images of the target class are labeled as **positive** (e.g., Angry = 1), while all other images (of all other expressions) are labeled as **negative** (e.g., not Angry = 0).

Pair (Classifier)
Angry Vs Not Angry
Sad Vs Not Sad
Neutral Vs Not Neutral
Fear Vs Not Fear
Surprise Vs Not Surprise
Happy Vs Not Happy

Support Vector Machine

SVM Model for Facial Expression Recognition: One-vs-Many approach

Example

The Angry classifier is trained to distinguish **Angry** images from any other expression (Fear, Happy, Sad, Neutral, and Surprise).

The Happy classifier is trained to distinguish **Happy** images from any other expression (Angry, Fear, Sad, Neutral, and Surprise), and so forth.

Pair (Classifier)
Angry Vs Not Angry
Sad Vs Not Sad
Neutral Vs Not Neutral
Fear Vs Not Fear
Surprise Vs Not Surprise
Happy Vs Not Happy

Support Vector Machine

SVM Model for Facial Expression Recognition: One-vs-Many approach

Testing New Image

- Each classifier independently decides whether the image belongs to **its specific class** (e.g., Angry or not Angry, Happy or not Happy).
- Each classifier produces a confidence score or distance from the decision boundary for its prediction:
 - A high positive score suggests a strong likelihood that the image belongs to the target class (e.g., the Angry classifier predicts a high score for an image it thinks is Angry).
 - A negative or lower score suggests the image does not belong to that class.

Pair (Classifier)
Angry Vs Not Angry
Sad Vs Not Sad
Neutral Vs Not Neutral
Fear Vs Not Fear
Surprise Vs Not Surprise
Happy Vs Not Happy

Support Vector Machine

SVM Model for Facial Expression Recognition: One-vs-Many approach

Selecting the Final Prediction

Each classifier will produce its score where we:

- Compare the confidence score from all classifiers.
- One with highest confidence score is the final predicted class.

Angry: -0.5

Fear: 0.2

Happy: -2.1

Sad: 2.8

Neutral: 0.5

Surprise: 0.1

Pair (Classifier)	
Angry Vs Not Angry	-0.5
Sad Vs Not Sad	2.8
Neutral Vs Not Neutral	0.5
Fear Vs Not Fear	0.2
Surprise Vs Not Surprise	0.1
Happy Vs Not Happy	-2.1

Here, **Sad** has the highest score (2.8), so the model classifies the image as **Sad**

Support Vector Machine

SVM Model for Facial Expression Recognition: One-vs-Many approach

Let's say we are predicting the expression for an image where the actual emotion is **"Sad."**

How??

Each classifier scores the image: The Angry classifier gives a low score because it doesn't detect typical Angry features.

The Sad classifier, seeing a eyebrows/lashes down or other "Sad" features, gives a high score.

Other classifiers (Surprise, Fear, etc.) might give neutral or low scores if their features don't align with the image.

Since the Sad classifier has the highest score, the model predicts Sad.

Support Vector Machine

SVM Model for Facial Expression Recognition: Comparison

Aspect	One-vs-All	One vs One
# of Classes	Same as of # of classes	Possible pairs of classes
Process	Highest confidence score amongst all the classifier	Voting
Training	Classifier see all the classes	See only two classes